

Contents

Temario — Claude Certified Architect: Foundations (CCA-F)	2
00 — Introducción	2
Qué es este temario	2
Aviso importante sobre la estructura	2
Perfil del candidato ideal (según el Exam Guide)	2
Cómo está construido cada dominio	2
Recursos generales de referencia	3
Cómo usar este temario junto al Exam Guide	3
1.1 El bucle agéntico (agentic loop)	3
1.2 Orquestación multiagente con patrón coordinador-subagentes	4
1.3 Invocación de subagentes, paso de contexto y spawning	4
1.4 Workflows multi-paso: enforcement programático vs guía por prompt	5
1.5 Hooks del Agent SDK para interceptar tools y normalizar datos	6
1.6 Estrategias de descomposición de tareas	6
1.7 Gestión de estado de sesión, resumido y bifurcación (forking)	7
Resumen de antipatrones del Dominio 1	7
Recursos oficiales para este dominio	7
2.1 Diseño de interfaces de tools con descripciones claras	8
2.2 Respuestas de error estructuradas en tools MCP	8
2.3 Distribución de tools entre agentes y configuración de <code>tool_choice</code>	9
2.4 Integración de servidores MCP en Claude Code y en flujos de agentes	10
2.5 Selección efectiva de tools nativas: <code>Read</code> , <code>Write</code> , <code>Edit</code> , <code>Bash</code> , <code>Grep</code> , <code>Glob</code>	10
Resumen de antipatrones del Dominio 2	11
Recursos oficiales para este dominio	11
3.1 Configuración de <code>CLAUDE.md</code> : jerarquía, alcance y organización modular	11
3.2 Comandos slash personalizados y skills	12
3.3 Reglas específicas por ruta (path-specific rules)	13
3.4 Plan mode vs ejecución directa	13
3.5 Refinamiento iterativo	14
3.6 Integración de Claude Code en pipelines CI/CD	14
Resumen de antipatrones del Dominio 3	15
Recursos oficiales para este dominio	15
4.1 Criterios explícitos para mejorar la precisión y reducir falsos positivos	15
4.2 Few-shot prompting para consistencia y calidad de salida	16
4.3 Salida estructurada forzada con <code>tool_use</code> y JSON Schema	16
4.4 Validación, reintento y bucles de feedback	17
4.5 Diseño de estrategias eficientes de procesamiento por lotes	17
4.6 Arquitecturas de revisión multi-instancia y multi-pasada	18
Resumen de antipatrones del Dominio 4	19
Recursos oficiales para este dominio	19
5.1 Gestión del contexto de conversación en interacciones largas	19
5.2 Patrones de escalado y resolución de ambigüedad	20
5.3 Estrategias de propagación de errores en sistemas multiagente	20
5.4 Gestión de contexto en exploración de código bases grandes	21
5.5 Diseño de flujos de revisión humana y calibración de confianza	22
5.6 Procedencia de la información y manejo de incertidumbre en síntesis multi-fuente	22
Resumen de antipatrones del Dominio 5	23
Recursos oficiales para este dominio	23
Bloque 1 — Las 12 preguntas oficiales (resumen)	23
Bloque 2 — Preguntas complementarias (dominios 4 y 5, y refuerzo de 1 y 2)	24
Cómo usar este archivo	26

Temario — Claude Certified Architect: Foundations (CCA-F)

00 — Introducción

Qué es este temario

Este documento desarrolla, dominio por dominio, todo el contenido que aparece resumido en la guía de estudio del **Claude Certified Architect – Foundations (CCA-F)**. Mientras que la guía de estudio te decía *qué* estudiar y *dónde*, este temario desarrolla *el contenido en sí*: los conceptos, la terminología, los patrones correctos y los antipatrones que el examen espera que reconozcas.

Está basado en: - El **Exam Guide oficial v1.0** (julio 2026) de Anthropic, que detalla los 5 dominios, sus task statements y el conocimiento/habilidades evaluados en cada uno - La documentación técnica oficial: docs.claude.com (Claude Code, Agent SDK), platform.claude.com/docs (Claude API), modelcontextprotocol.io (MCP) - El contenido de los 4 cursos del Claude Partner Network Learning Path: *Introduction to Agent Skills*, *Building with the Claude API*, *Introduction to Model Context Protocol* y *Claude Code in Action*

Aviso importante sobre la estructura

El Exam Guide oficial define **5 dominios**, no 4. Este temario los cubre todos:

Archivo	Dominio	Peso en el examen
01	Agentic Architecture & Orchestration	27%
02	Tool Design & MCP Integration	18%
03	Claude Code Configuration & Workflows	20%
04	Prompt Engineering & Structured Output	20%
05	Context Management & Reliability	15%
06	Preguntas tipo	—

Omitir el Dominio 5 dejaría sin cubrir un 15% del examen, así que se incluye como archivo independiente aunque no estuviera en la estructura inicial solicitada.

Perfil del candidato ideal (según el Exam Guide)

Anthropic describe al candidato ideal como un arquitecto de soluciones con experiencia práctica de **6 meses o más** construyendo con Claude, que ya ha trabajado con: - El Agent SDK: orquestación multiagente, delegación a subagentes, integración de tools, hooks de ciclo de vida - Claude Code: **CLAUDE.md**, Agent Skills, integración de servidores MCP, plan mode - Diseño de interfaces de tools y recursos MCP para conectar con sistemas backend - Prompt engineering para salida estructurada fiable (JSON schemas, few-shot, extracción) - Gestión de contexto en documentos largos, conversaciones multi-turno y trasposos entre agentes - Integración de Claude en pipelines CI/CD - Decisiones de escalado, manejo de errores y patrones de auto-evaluación

Si notas que alguno de estos puntos te resulta ajeno, es una señal de qué dominio necesita más tiempo de estudio antes del examen.

Cómo está construido cada dominio

Cada archivo de dominio sigue esta estructura: 1. **Resumen y peso** — qué cubre y cuánto pesa en el examen 2. **Desarrollo de cada task statement** — el “conocimiento de” y “habilidades en” que exige el Exam Guide, explicado con contexto, ejemplos y razonamiento (no solo la lista de puntos) 3. **Antipatrones frecuentes** — qué es lo que el examen espera que reconozcas como *incorrecto*, porque buena parte de las preguntas están diseñadas alrededor de identificar la opción con un fallo sutil, no la más obviamente mala 4. **Recursos oficiales** — enlaces a la documentación y curso concretos donde ampliar

Recursos generales de referencia

- Claude Code: <https://docs.claude.com/en/docs/claude-code>
- Claude Agent SDK: <https://docs.claude.com/en/api/agent-sdk/overview>
- Claude API (Developer Guide): <https://platform.claude.com/docs>
- Model Context Protocol: <https://modelcontextprotocol.io>
- Índice completo de documentación (útil para explorar por tu cuenta): <https://code.claude.com/docs/llms.txt>

Cómo usar este temario junto al Exam Guide

El Exam Guide es el documento *normativo*: define exactamente qué se evalúa y con qué peso. Este temario es *explicativo*: desarrolla ese contenido para que lo entiendas en profundidad, no solo lo memorices. La recomendación es:

1. Lee el dominio en este temario
2. Practica el concepto en un entorno real (Claude Code, un proyecto con Agent SDK, un servidor MCP de prueba)
3. Repasa el resumen de “antipatrones” de ese dominio
4. Pasa a las preguntas tipo relacionadas con ese dominio (archivo 06)

Nota: dado que el Exam Guide es contenido confidencial y propietario de Anthropic, este temario parafrasea y desarrolla sus conceptos en vez de reproducirlo literalmente. Para el texto exacto de cada task statement, consulta el PDF oficial. # 01 — Dominio 1: Agentic Architecture & Orchestration (27%)

Es el dominio con más peso del examen. Cubre cómo funciona el bucle de un agente por dentro, cómo se orquestan varios agentes entre sí, cómo se controla su comportamiento con hooks, y cómo se gestiona el estado entre sesiones largas.

Recursos principales: curso *Introduction to Agent Skills*, curso *Claude Code in Action*, documentación del Agent SDK (<https://docs.claude.com/en/api/agent-sdk/overview>), guía *Subagents in the SDK* del Developer Guide.

1.1 El bucle agéntico (agentic loop)

El concepto

Un agente construido sobre la API de Claude no es más que un bucle: se envía una petición al modelo, el modelo decide si necesita usar una herramienta (`tool_use`) o si ya puede dar una respuesta final (`end_turn`), se ejecuta la herramienta si corresponde, y el resultado se reincorpora a la conversación para la siguiente vuelta del bucle.

La señal que gobierna todo esto es el campo `stop_reason` de la respuesta: - `tool_use` → el modelo ha pedido ejecutar una o más herramientas. El código debe ejecutarlas y devolver los resultados como nuevos mensajes de tipo `tool_result`. - `end_turn` → el modelo considera que ha terminado y no necesita más herramientas. Aquí el bucle debe parar.

Este patrón es la base tanto de la API “cruda” (Messages API con `tool_use`) como de todo lo que construyen el Agent SDK y Claude Code por encima.

Por qué importa cómo se gestiona el historial

Cada resultado de herramienta se añade al historial de la conversación, no se descarta. Esto es lo que permite que el modelo “recuerde” lo que ha encontrado en pasos anteriores y razone sobre ello en el siguiente. Si el código que implementa el bucle no reinyecta los `tool_result` en el array de mensajes antes de la siguiente llamada, el agente pierde continuidad y puede repetir acciones o contradecirse.

Antipatrones que el examen espera que reconozcas

Este es uno de los puntos donde el Exam Guide es muy explícito sobre qué **no** hacer: - **Parsear el texto de la respuesta buscando frases** (“ya he terminado”, “no necesito más herramientas”) para decidir si el bucle

continúa. Es frágil: el modelo no sigue un formato fijo en su texto libre. - **Poner un límite arbitrario de iteraciones** como mecanismo *principal* de parada. Un límite de iteraciones es una salvaguarda razonable como red de seguridad, pero no debe ser la lógica que decide cuándo el agente ha terminado — para eso está `stop_reason`. - **Comprobar si hay contenido de texto** en la respuesta del asistente como señal de finalización. Un modelo puede escribir texto explicativo (razonamiento intermedio) y *aun así* pedir una tool en la misma respuesta.

La forma correcta es siempre: continuar mientras `stop_reason == "tool_use"`, terminar cuando `stop_reason == "end_turn"`.

1.2 Orquestación multiagente con patrón coordinador-subagentes

El patrón hub-and-spoke

Cuando una tarea es demasiado grande o heterogénea para un solo agente (por ejemplo, un sistema de investigación que necesita buscar en la web, analizar documentos y redactar un informe), el patrón recomendado es un **coordinador** que actúa como centro (*hub*) y varios **subagentes** especializados como radios (*spoke*).

Puntos clave de este patrón: - **Toda la comunicación entre subagentes pasa por el coordinador**. Los subagentes no se hablan directamente entre sí. Esto centraliza la observabilidad, el manejo de errores y el enrutamiento de información. - El coordinador decide **qué subagentes invocar** según la complejidad de la consulta — no todas las consultas necesitan pasar por el pipeline completo. - El coordinador es responsable de la **descomposición de la tarea**, la **delegación**, y la **agregación de resultados** al final.

El riesgo de una descomposición demasiado estrecha

Un fallo típico de este patrón (y que aparece explícitamente como pregunta de examen) es que el coordinador descomponga el tema en subtarefas que no cubren todo el alcance pedido. Por ejemplo, si se le pide investigar “el impacto de la IA en las industrias creativas” y el coordinador solo genera subtarefas sobre artes visuales (arte digital, diseño gráfico, fotografía), el informe final quedará incompleto aunque cada subagente haya hecho bien su trabajo — porque el problema está en el reparto de tareas, no en la ejecución.

Refinamiento iterativo

Un coordinador bien diseñado no se limita a lanzar subagentes una vez y agregar lo que devuelvan. Debe: 1. Evaluar la salida de síntesis en busca de huecos de cobertura 2. Re-delegar a los subagentes de búsqueda/análisis con consultas más específicas dirigidas a esos huecos 3. Volver a invocar la síntesis con la información ampliada, repitiendo hasta que la cobertura sea suficiente

1.3 Invocación de subagentes, paso de contexto y spawning

La tool Task

En el Agent SDK, la forma mecánica de que un coordinador cree un subagente es a través de la **tool Task**. Esto tiene una implicación de configuración muy concreta que aparece en el examen: para que un agente coordinador pueda invocar subagentes, su lista de `allowedTools` **debe incluir "Task"** explícitamente. Si no está, el coordinador no podrá delegar aunque el resto de la configuración sea correcta.

Los subagentes no heredan contexto automáticamente

Este es uno de los puntos conceptuales más importantes del dominio: **un subagente no ve la conversación del coordinador por defecto**. No hay memoria compartida implícita entre invocaciones. Todo lo que el subagente necesite saber — hallazgos previos, resultados de búsquedas anteriores, instrucciones específicas — tiene que estar **explícitamente incluido en el prompt** con el que se invoca.

Esto significa que, si un subagente de síntesis necesita los resultados de un subagente de búsqueda web y otro de análisis de documentos, esos resultados completos deben incluirse literalmente en el prompt del subagente de síntesis. No basta con decir “usa lo que encontraron los otros agentes”.

Separar contenido de metadatos

Cuando se pasa contexto entre agentes, es una buena práctica usar formatos de datos estructurados que separen el **contenido** (el hallazgo en sí) de los **metadatos** (URL de la fuente, nombre del documento, número de página). Esto preserva la atribución de las fuentes a lo largo de toda la cadena de agentes, algo crítico si el resultado final necesita citar de dónde salió cada afirmación.

AgentDefinition y ejecución en paralelo

Cada tipo de subagente se configura mediante una **AgentDefinition**, que incluye su descripción, su system prompt propio y las restricciones de tools que tendrá disponibles (ver Dominio 2 para el porqué de restringir tools por agente).

Para lanzar subagentes en paralelo, el coordinador debe emitir **varias llamadas a la tool Task en una sola respuesta**, no una por turno separado. Lanzarlos en turnos sucesivos los ejecuta secuencialmente, perdiendo la ventaja de paralelismo.

Prompts orientados a objetivos, no a procedimientos

Al escribir el prompt del coordinador para los subagentes, es preferible especificar **metas de investigación y criterios de calidad** en lugar de instrucciones procedimentales paso a paso. Esto da al subagente margen para adaptarse a lo que encuentra, en vez de forzarlo a seguir una receta rígida que puede no encajar con la información real disponible.

Sesiones fork

`fork_session` permite crear una rama independiente de una sesión ya existente para explorar un enfoque distinto partiendo de la misma base de análisis — por ejemplo, comparar dos estrategias de refactorización partiendo del mismo entendimiento inicial del código.

1.4 Workflows multi-paso: enforcement programático vs guía por prompt

La diferencia clave

Hay dos formas de asegurar que un agente siga un orden concreto de pasos: - **Enforcement programático**: hooks o barreras (*gates*) en el código que bloquean físicamente una acción hasta que se cumple un prerequisite. - **Guía por prompt**: simplemente decirle al modelo en sus instrucciones que siga un orden determinado.

La guía por prompt tiene una tasa de fallo **no nula**: por muy bien redactada que esté la instrucción, un modelo puede saltársela ocasionalmente. Cuando el cumplimiento debe ser determinista — por ejemplo, verificar la identidad de un cliente antes de procesar un reembolso — la única opción fiable es el enforcement programático.

Ejemplo de gate programático

Bloquear la tool `process_refund` hasta que `get_customer` haya devuelto un ID de cliente verificado es un ejemplo directo de esto: la barrera no depende de que el modelo “recuerde” verificar antes de reembolsar, sino de que el código simplemente no permite ejecutar `process_refund` sin ese prerequisite cumplido.

Descomposición de solicitudes con varios asuntos

Cuando una solicitud de un usuario contiene varios problemas distintos (por ejemplo, un cliente que pregunta por un reembolso y también reporta un cargo duplicado), conviene descomponer la solicitud en ítems separados, investigar cada uno usando un contexto compartido, y luego sintetizar una única resolución unificada — en vez de tratarlo como una sola pregunta ambigua o resolver cada parte de forma completamente aislada.

Trasposos (handoffs) estructurados a humanos

Cuando un caso se escala a un agente humano, ese humano normalmente no tiene acceso a la transcripción completa de la conversación. Por eso, el trasposo debe compilarse como un resumen estructurado: ID del cliente, causa raíz del problema, importe implicado, acción recomendada. Un simple “escalado, revisar conversación” no es suficiente.

1.5 Hooks del Agent SDK para interceptar tools y normalizar datos

Qué son los hooks

Los hooks son puntos de intercepción programática en el ciclo de vida del agente. El más relevante para este dominio es **PostToolUse**, que se ejecuta después de que una tool devuelve su resultado, pero antes de que ese resultado llegue al modelo.

Dos usos principales

1. **Normalización de datos heterogéneos.** Si distintas tools MCP devuelven fechas en formatos distintos (timestamps Unix, ISO 8601) o códigos de estado numéricos con significados distintos, un hook **PostToolUse** puede normalizar todo a un formato común antes de que el agente tenga que razonar sobre ello. Esto evita que el modelo tenga que “adivinar” el significado de un formato inconsistente.
2. **Interceptación de llamadas salientes para cumplir reglas de negocio.** Un hook puede bloquear una llamada a una tool si viola una política — por ejemplo, bloquear un reembolso que supere los 500 \$ y redirigir automáticamente el flujo hacia una escalada humana, en lugar de confiar en que el prompt del sistema disuada al modelo de aprobar ese reembolso.

El criterio para elegir hooks sobre prompts

La regla general que evalúa el examen: si una regla de negocio necesita **garantía determinista** de cumplimiento, se implementa como hook. Si es una preferencia o guía que tolera cierto margen de error, basta con incluirla en el prompt.

1.6 Estrategias de descomposición de tareas

Cadena de prompts (prompt chaining) vs descomposición dinámica

- **Prompt chaining:** una secuencia fija y predecible de pasos. Útil cuando la tarea tiene una estructura conocida de antemano — por ejemplo, revisar cada archivo de un pull request de forma individual y luego hacer una pasada de integración que compare los archivos entre sí.
- **Descomposición dinámica:** el plan de subtarefas se genera sobre la marcha, en función de lo que se va descubriendo. Es la opción adecuada para tareas abiertas donde no se puede predecir de antemano cuántos pasos harán falta ni cuáles — por ejemplo, “añade tests exhaustivos a este código heredado”, donde primero hay que mapear la estructura del proyecto antes de saber qué áreas necesitan más atención.

Por qué dividir revisiones grandes en pasadas separadas

Cuando una revisión de código abarca muchos archivos a la vez, el modelo tiende a diluir su atención: da respuestas detalladas para algunos archivos y superficiales para otros, o incluso contradice en un archivo lo que aprobó en otro con un patrón idéntico. La solución no es cambiar a un modelo con ventana de contexto más grande — eso no arregla la dilución de atención — sino **dividir la revisión en pasadas más pequeñas**: una pasada de análisis local por archivo, más una pasada de integración aparte centrada en el flujo de datos entre archivos.

Descomposición adaptativa de tareas abiertas

Para tareas como “añade tests a este código heredado”, el enfoque recomendado es: primero mapear la estructura general, después identificar las áreas de mayor impacto, y finalmente construir un plan priorizado que se adapte a medida que se descubren nuevas dependencias — no intentar planificar todo de antemano con información incompleta.

1.7 Gestión de estado de sesión, resumido y bifurcación (forking)

Resumido de sesiones con nombre

`--resume <nombre-de-sesión>` permite retomar una conversación anterior identificada por nombre. Es útil para continuar una investigación concreta entre distintas sesiones de trabajo sin perder el contexto acumulado.

Cuándo resumir y cuándo empezar de cero

No siempre conviene resumir una sesión antigua. Si el código ha cambiado desde el análisis anterior, los resultados de herramientas guardados en esa sesión pueden estar obsoletos (“stale”). En ese caso, es más fiable **empezar una sesión nueva inyectando un resumen estructurado** de lo relevante, que intentar resumir una sesión cuyo contexto ya no refleja el estado real del código.

Si se retoma una sesión después de haber modificado archivos que ya se habían analizado, hay que informar explícitamente al agente de qué archivos cambiaron, para que haga un reanálisis dirigido de esos archivos concretos — en lugar de forzarlo a re-explorar todo el proyecto desde cero.

fork_session

Permite crear ramas independientes de exploración a partir de una misma base de análisis compartida — por ejemplo, para comparar dos enfoques de refactorización distintos partiendo del mismo punto de entendimiento del código, sin que un enfoque contamine el contexto del otro.

Resumen de antipatrones del Dominio 1

Antipatrón	Por qué falla	Alternativa correcta
Parsear texto libre para saber si el bucle terminó	Frágil, el formato no es fiable	Usar <code>stop_reason</code>
Límite de iteraciones como mecanismo principal de parada	No refleja si la tarea está realmente completa	<code>stop_reason</code> + límite solo como red de seguridad
Confiar solo en el prompt para un orden crítico de pasos	Tasa de fallo no nula en casos de alto impacto	Hooks / gates programáticos
Asumir que un subagente “sabe” lo que hizo otro agente	No hay memoria compartida implícita	Pasar el contexto explícitamente en el prompt
Aumentar el tamaño del modelo/contexto para arreglar revisiones inconsistentes	No corrige la dilución de atención	Dividir en pasadas por archivo + integración
Descomposición del coordinador demasiado estrecha	Dejan fuera partes del alcance pedido	Verificar que las subtareas cubren todo el dominio de la pregunta

Recursos oficiales para este dominio

- Agent SDK — visión general: <https://docs.claude.com/en/api/agent-sdk/overview>
- Subagents in the SDK: sección “Guides” del Developer Guide en <https://platform.claude.com/docs>
- Session Management (guía del SDK)

- Curso 1.1 *Introduction to Agent Skills* y curso 1.4 *Claude Code in Action* del Partner Network Learning Path # 02 — Dominio 2: Tool Design & MCP Integration (18%)

Este dominio trata sobre cómo diseñar bien las herramientas que un agente puede usar, cómo comunicar errores de forma que el agente pueda actuar con inteligencia, cómo repartir tools entre varios agentes, y cómo integrar servidores MCP en Claude Code y en flujos de agentes.

Recursos principales: curso *Introduction to Model Context Protocol*, especificación oficial en modelcontextprotocol.io, documentación de tool use en platform.claude.com/docs.

2.1 Diseño de interfaces de tools con descripciones claras

La descripción es el mecanismo de selección

Este es el punto de partida conceptual de todo el dominio: **la descripción de una tool es el principal mecanismo que usa el modelo para decidir cuál usar**. No hay una capa de “razonamiento oculto” adicional — si dos tools tienen descripciones mínimas y parecidas, el modelo tendrá una tasa de selección poco fiable entre ellas, incluso si internamente hacen cosas completamente distintas.

Qué debe llevar una buena descripción

- Formatos de entrada esperados
- Ejemplos de consultas típicas
- Casos límite
- Explicación explícita de los límites: cuándo usar esta tool y cuándo usar una alternativa parecida

El problema de las tools solapadas

Un ejemplo característico que aparece en el examen: dos tools, `analyze_content` y `analyze_document`, con descripciones casi idénticas (“analiza contenido” / “analiza documentos”). El modelo las confunde sistemáticamente. La solución no es añadir más ejemplos few-shot como parche — la causa raíz es que las descripciones no diferencian el propósito real de cada tool. La corrección correcta pasa por: - **Renombrar** las tools para que el nombre ya sea distintivo (por ejemplo, `analyze_content` → `extract_web_results`, con una descripción específica para contenido web) - **Dividir una tool genérica en varias tools de propósito específico** con contratos de entrada/salida bien definidos — por ejemplo, dividir un `analyze_document` genérico en `extract_data_points`, `summarize_content` y `verify_claim_against_source`

El system prompt también puede sabotear la selección de tools

Instrucciones del system prompt sensibles a palabras clave pueden crear asociaciones no intencionadas con una tool concreta, incluso cuando las descripciones de las tools están bien escritas. Por eso, al depurar un problema de selección de tools, hay que revisar tanto las descripciones de las tools **como** el propio system prompt.

2.2 Respuestas de error estructuradas en tools MCP

El flag `isError`

El protocolo MCP define un patrón estándar para comunicar que una tool ha fallado: el flag `isError`. Pero el flag por sí solo no basta — lo importante es *qué información acompaña* a ese error.

Categorías de error

El examen distingue explícitamente cuatro tipos de error que una tool puede encontrar: - **Transitorio** — timeouts, servicio no disponible temporalmente - **Validación** — entrada inválida - **Negocio** — violación de una política (por ejemplo, un reembolso que excede el límite permitido) - **Permiso** — el llamante no tiene autorización

Por qué un error genérico es un problema

Una respuesta de error uniforme tipo "Operation failed" no le da al agente ninguna base para decidir cómo reaccionar: ¿reintenta?, ¿se disculpa con el usuario?, ¿escala? La forma correcta es devolver **metadatos estructurados**: - **errorCategory** (transient / validation / permission / business) - **isRetryable** (booleano) - Una descripción legible por humanos

Cuando el error es de negocio y no reintentable, conviene incluir además un **retryable: false** junto con una explicación orientada al cliente, para que el agente pueda comunicar el motivo con claridad en vez de simplemente fallar.

Errores locales vs propagados

Cuando un subagente encuentra un error transitorio, lo ideal es que intente **recuperación local** primero (reintentar internamente), y solo propague al coordinador los errores que no ha podido resolver por sí mismo — acompañados del resultado parcial obtenido y de lo que ya se intentó, para que el coordinador no repita el mismo intento fallido.

Un matiz importante: fallo de acceso vs resultado vacío legítimo

Un timeout o un fallo de conexión (que exige decidir si reintentar) es conceptualmente distinto de una consulta que se ejecutó correctamente pero **no encontró resultados** (un resultado vacío legítimo). Confundir ambos casos en la misma categoría de “sin resultado” hace que el agente no pueda distinguir cuándo debe insistir y cuándo simplemente no hay nada que encontrar.

2.3 Distribución de tools entre agentes y configuración de tool_choice

Menos tools por agente, mejor selección

Dar a un agente acceso a demasiadas tools (el Exam Guide pone el ejemplo de 18 tools en vez de 4-5) degrada la fiabilidad de selección, porque aumenta la complejidad de la decisión en cada turno. La recomendación es **acceso de tools acotado por rol**: cada agente solo tiene las tools relevantes para su especialización, con alguna tool cruzada puntual solo cuando hay una necesidad de alta frecuencia que lo justifique.

El riesgo de tools fuera de la especialización de un agente

Un agente que tiene acceso a tools fuera de su rol tiende a usarlas mal — el ejemplo típico es un agente de síntesis con acceso a búsqueda web que empieza a hacer sus propias búsquedas en lugar de limitarse a sintetizar lo que ya le han entregado los agentes de investigación.

Solución intermedia: tools cruzadas acotadas

En vez de dar acceso total o ningún acceso, se puede dar a un agente una tool específica y acotada para una necesidad frecuente concreta — por ejemplo, una tool **verify_fact** para el agente de síntesis, que le permita verificar hechos puntuales sin tener que devolver el control al coordinador para casos simples, mientras que los casos complejos de verificación siguen enrutándose a través del coordinador hacia el agente de búsqueda.

Opciones de tool_choice

- "auto" — el modelo decide libremente si llama a una tool o responde con texto
- "any" — el modelo debe llamar a alguna tool, pero puede elegir cuál (útil para garantizar que siempre haya salida estructurada cuando hay varios schemas posibles y no se sabe de antemano cuál aplica)
- Selección forzada ({**"type": "tool", "name": "..."**}) — el modelo debe llamar exactamente a la tool indicada. Útil para forzar un orden, por ejemplo forzar **extract_metadata** antes de pasar a tools de enriquecimiento, procesando los siguientes pasos en turnos posteriores

2.4 Integración de servidores MCP en Claude Code y en flujos de agentes

Alcance del servidor: proyecto vs usuario

- `.mcp.json` a nivel de proyecto — para tooling compartido por todo el equipo, versionado junto con el repositorio
- `~/.claude.json` a nivel de usuario — para servidores personales o experimentales que no se quieren compartir con el equipo

Gestión de credenciales

`.mcp.json` admite **expansión de variables de entorno** (por ejemplo `${GITHUB_TOKEN}`), lo que permite compartir la configuración del servidor sin comprometer las credenciales reales en el control de versiones.

Descubrimiento simultáneo de tools

Todas las tools de todos los servidores MCP configurados se descubren en el momento de la conexión y quedan disponibles simultáneamente para el agente. Esto tiene implicaciones directas con el punto 2.3: cuantos más servidores MCP se conecten sin criterio, más tools compiten por la atención del modelo en cada decisión.

MCP resources

Además de las tools (que ejecutan acciones), MCP define **resources**: un mecanismo para exponer catálogos de contenido — resúmenes de issues, jerarquías de documentación, esquemas de base de datos — de forma que el agente tenga visibilidad sobre qué datos existen **sin tener que hacer llamadas exploratorias** a tools solo para descubrir qué hay disponible.

Cuándo usar servidores comunitarios vs construir uno propio

Para integraciones estándar (por ejemplo, Jira), es preferible usar un servidor MCP de la comunidad ya existente antes que construir uno propio. Reservar el desarrollo de servidores MCP a medida para flujos de trabajo realmente específicos del equipo que no tienen equivalente estándar.

Un matiz de adopción: describir bien las tools MCP frente a las tools nativas

Si las descripciones de las tools de un servidor MCP son pobres, el agente puede preferir tools nativas más simples (como `Grep`) aunque la tool MCP sea objetivamente más capaz para esa tarea. La solución es enriquecer las descripciones de las tools MCP explicando en detalle sus capacidades y salidas — no es un problema del modelo “prefiriendo lo conocido”, es, otra vez, un problema de descripción insuficiente.

2.5 Selección efectiva de tools nativas: Read, Write, Edit, Bash, Grep, Glob

Cuándo usar cada una

- **Grep** — búsqueda de contenido dentro de archivos: nombres de funciones, mensajes de error, sentencias de import. Es la tool correcta para “encontrar dónde se usa X” o “encontrar todos los sitios que llaman a esta función”.
- **Glob** — búsqueda de rutas de archivo por patrón de nombre (por ejemplo, `**/*.test.tsx`). No busca contenido, busca nombres/rutas.
- **Read / Write** — operaciones de archivo completo: cargar el archivo entero o escribirlo entero.
- **Edit** — modificaciones puntuales, basadas en encontrar un fragmento de texto único dentro del archivo y reemplazarlo.

Cuándo Edit falla y qué hacer

Si el texto que se quiere reemplazar con `Edit` no es único dentro del archivo (aparece en varios sitios), la operación falla porque no puede identificar de forma inequívoca dónde aplicar el cambio. En ese caso, el patrón de repliegue

fiable es usar **Read seguido de Write**: leer el archivo completo, construir la nueva versión completa, y escribirla entera.

Cómo construir entendimiento de un código base de forma incremental

El patrón recomendado para explorar un proyecto grande no es leer todos los archivos de golpe: 1. Empezar con **Grep** para localizar puntos de entrada (por ejemplo, dónde se define una función clave) 2. Usar **Read** para seguir los imports desde ahí y trazar el flujo real de datos/llamadas 3. Para rastrear el uso de una función a través de módulos wrapper, primero identificar todos los nombres exportados relacionados, y luego buscar cada uno de esos nombres por separado en el código base

Resumen de antipatrones del Dominio 2

Antipatrón	Por qué falla	Alternativa correcta
Descripciones de tools mínimas o casi idénticas	El modelo no puede diferenciarlas de forma fiable	Descripciones detalladas con límites explícitos, o renombrar/dividir tools
Mensajes de error genéricos (“Operation failed”)	El agente no puede decidir cómo reaccionar	Metadatos estructurados: categoría, isRetryable , descripción
Dar a un agente todas las tools disponibles	Degrada la fiabilidad de selección	Acceso acotado por rol, con tools cruzadas puntuales si hace falta
Tratar un resultado vacío como fallo de acceso (o viceversa)	El coordinador no puede decidir bien qué hacer	Distinguir explícitamente ambos casos en la respuesta
Construir un servidor MCP propio para una integración estándar	Reinventa lo que ya existe y mantiene, con más coste	Usar servidores MCP comunitarios cuando existan
Añadir few-shot para arreglar confusión de tools con descripciones pobres	No corrige la causa raíz	Mejorar/dividir las descripciones primero

Recursos oficiales para este dominio

- Model Context Protocol — especificación: <https://modelcontextprotocol.io>
- Tool use (Claude API): sección “Tools” del Developer Guide en <https://platform.claude.com/docs>
- MCP in the SDK / MCP connector / Remote MCP servers: Developer Guide, sección “MCP in the API” y “Guides”
- Curso 1.3 *Introduction to Model Context Protocol # 03* — Dominio 3: Claude Code Configuration & Workflows (20%)

Este dominio cubre cómo configurar Claude Code para un equipo real: jerarquía de configuración, comandos y skills personalizados, reglas condicionales, plan mode, refinamiento iterativo, e integración en CI/CD.

Recursos principales: curso *Claude Code in Action*, curso *Introduction to Agent Skills*, documentación en <https://docs.claude.com/en/docs/claude-code>.

3.1 Configuración de CLAUDE.md: jerarquía, alcance y organización modular

Los tres niveles de jerarquía

- **Nivel usuario** — `~/.claude/CLAUDE.md`. Se aplica solo a ese usuario en su máquina. **No se comparte con el equipo** vía control de versiones.
- **Nivel proyecto** — `.claude/CLAUDE.md` o un `CLAUDE.md` en la raíz del repositorio. Se comparte con todo el equipo porque vive dentro del repositorio.
- **Nivel directorio** — archivos `CLAUDE.md` en subdirectorios concretos, para convenciones específicas de esa parte del proyecto.

Un fallo de diagnóstico típico

Si un nuevo miembro del equipo no recibe ciertas instrucciones que “todo el mundo tiene”, la causa más probable es que esas instrucciones estén guardadas en la configuración de **nivel usuario** de otra persona (que nunca se comparte) en lugar de en la configuración de **nivel proyecto** (que sí viaja con el repositorio). Es un error de ubicación, no de contenido.

@import para mantener la configuración modular

En vez de un único `CLAUDE.md` monolítico, se pueden referenciar archivos externos con la sintaxis `@import`. Esto permite, por ejemplo, que cada paquete de un monorepo importe solo los archivos de estándares relevantes para su propio dominio, en lugar de cargar un documento gigante con reglas que no le aplican.

.claude/rules/ como alternativa a un CLAUDE.md monolítico

Cuando el archivo de configuración crece demasiado y mezcla temas distintos (testing, convenciones de API, despliegue), conviene dividirlo en archivos temáticos independientes dentro de `.claude/rules/` — por ejemplo, `testing.md`, `api-conventions.md`, `deployment.md` — en lugar de mantener todo en un único documento difícil de mantener.

El comando `/memory`

Sirve para verificar qué archivos de memoria (`CLAUDE.md`, reglas, etc.) se han cargado realmente en la sesión actual. Es la herramienta de diagnóstico correcta cuando el comportamiento de Claude Code es inconsistente entre sesiones o entre miembros del equipo — permite confirmar si el archivo esperado se está cargando o no.

3.2 Comandos slash personalizados y skills

Alcance de los comandos: proyecto vs usuario

- `.claude/commands/` — comandos compartidos con todo el equipo vía control de versiones. Si se quiere que un comando (por ejemplo `/review`) esté disponible para cualquiera que clone o haga `pull` del repositorio, este es el lugar correcto.
- `~/.claude/commands/` — comandos personales, no compartidos.

Skills en `.claude/skills/`

Cada skill se define con un archivo `SKILL.md` que admite configuración en el frontmatter: - **context: fork** — ejecuta la skill en un contexto de sub-agente aislado, de forma que su salida (potencialmente muy verbosa, como un análisis exhaustivo de código, o exploratoria, como una lluvia de ideas de alternativas) no contamina la conversación principal. - **allowed-tools** — restringe qué tools puede usar la skill durante su ejecución (por ejemplo, limitarla solo a operaciones de escritura de archivo, para evitar acciones destructivas no deseadas). - **argument-hint** — sugiere al desarrollador qué parámetros debe pasar cuando invoca la skill sin argumentos.

Skills personales sin afectar al equipo

Es posible crear variantes personales de una skill en `~/.claude/skills/`, con un nombre distinto al de la skill compartida, para experimentar sin afectar a los demás miembros del equipo que usan la versión oficial.

Skills vs CLAUDE.md: cuándo usar cada uno

- **CLAUDE.md** — para estándares universales que deben cargarse siempre, en cada sesión.
- **Skills** — para flujos de trabajo específicos de una tarea, que se invocan bajo demanda solo cuando hacen falta.

3.3 Reglas específicas por ruta (path-specific rules)

El problema que resuelven

Un `CLAUDE.md` por subdirectorio funciona bien cuando las convenciones coinciden con la estructura de carpetas. Pero hay convenciones que **no** siguen esa estructura — por ejemplo, archivos de test que están distribuidos junto al código que prueban, repartidos por todo el proyecto (`Button.test.tsx` junto a `Button.tsx`, `Api.test.ts` junto a `Api.ts`, etc.), en vez de agrupados en una carpeta `tests/`.

La solución: frontmatter YAML con patrones glob

Los archivos dentro de `.claude/rules/` pueden llevar un campo `paths` en su frontmatter YAML con patrones glob (por ejemplo `paths: ["**/*.test.tsx"]`). La regla se carga **solo** cuando se está editando un archivo que coincide con ese patrón, sin importar en qué carpeta esté — lo que reduce contexto irrelevante y consumo de tokens comparado con cargar siempre todo.

Por qué esto es mejor que un `CLAUDE.md` por directorio en este caso concreto

Un `CLAUDE.md` está atado a un directorio; no puede “seguir” a archivos de un mismo tipo dispersos por todo el árbol del proyecto. Las reglas por patrón glob sí pueden, porque se activan por el nombre/ruta del archivo, no por su ubicación jerárquica.

3.4 Plan mode vs ejecución directa

Cuándo usar plan mode

Plan mode está pensado para tareas que implican: - Cambios a gran escala - Múltiples enfoques válidos entre los que elegir - Decisiones arquitectónicas - Modificaciones que tocan muchos archivos

Ejemplos típicos: reestructurar un monolito en microservicios, una migración de librería que afecta a decenas de archivos, o elegir entre dos enfoques de integración con requisitos de infraestructura distintos.

Cuándo usar ejecución directa

Para cambios simples y bien acotados: un fix de un solo archivo con una traza de error clara, añadir una comprobación de validación puntual a una función concreta.

Por qué importa la diferencia

Plan mode permite explorar y diseñar de forma segura **antes** de comprometerse a hacer cambios, evitando el coste de tener que deshacer trabajo cuando se descubren dependencias a mitad de la implementación. Usar ejecución directa en una tarea que en realidad tenía implicaciones arquitectónicas es un patrón de riesgo — el trabajo puede tener que rehacerse cuando aparecen las dependencias que no se habían explorado de antemano.

El subagente Explore

Para fases de descubrimiento muy verbosas dentro de una tarea más grande, se puede usar un subagente **Explore** que aísla ese ruido y devuelve solo un resumen a la conversación principal — evitando que la exploración agote la ventana de contexto de la sesión principal.

Combinar ambos modos

Un patrón habitual y correcto es usar plan mode para la fase de investigación/diseño (por ejemplo, planificar una migración de librería) y luego cambiar a ejecución directa para implementar el plan ya decidido.

3.5 Refinamiento iterativo

Ejemplos concretos de entrada/salida

Cuando una descripción en prosa produce resultados inconsistentes, la técnica más efectiva no es “explicar mejor con más palabras” sino dar **2-3 ejemplos concretos de entrada/salida** que muestren exactamente la transformación esperada.

Desarrollo guiado por tests (TDD) como técnica de iteración

Escribir primero la batería de tests que cubra el comportamiento esperado, los casos límite y los requisitos de rendimiento, e iterar compartiendo los fallos de esos tests para guiar la mejora progresiva del código — en vez de iterar solo con descripciones verbales de lo que está mal.

El patrón de entrevista

Consiste en pedirle a Claude que **haga preguntas** antes de implementar, para sacar a la luz consideraciones de diseño que el desarrollador podría no haber anticipado (por ejemplo, estrategias de invalidación de caché, o modos de fallo) — especialmente valioso en dominios poco familiares para quien está pidiendo el cambio.

Casos de prueba específicos para arreglar edge cases

Si hay un problema concreto (por ejemplo, manejo incorrecto de valores nulos en un script de migración), aportar un caso de prueba específico con entrada y salida esperada es más efectivo que una descripción general del problema.

Un mensaje detallado vs iteración secuencial

- Si varios problemas **interactúan entre sí** (arreglar uno afecta a cómo se debe arreglar otro), es mejor describirlos todos juntos en un único mensaje detallado.
 - Si los problemas son **independientes**, iterar secuencialmente arreglando uno cada vez es preferible.
-

3.6 Integración de Claude Code en pipelines CI/CD

Modo no interactivo

El flag **-p** (o **--print**) es la forma documentada de ejecutar Claude Code sin esperar entrada interactiva — imprescindible en un pipeline automatizado, donde no hay nadie para responder a un prompt. Sin este flag, un job de CI puede quedarse colgado esperando input que nunca llega.

Salida estructurada para CI

- **--output-format json** junto con **--json-schema** fuerzan una salida que un script puede parsear de forma fiable, por ejemplo para publicar comentarios automáticos en un pull request.

CLAUDE.md como fuente de contexto para revisiones automatizadas

Documentar en **CLAUDE.md** los estándares de testing, las convenciones de fixtures disponibles y los criterios de revisión mejora directamente la calidad de lo que Claude Code genera o revisa en el pipeline — reduce comentarios de bajo valor y sugerencias de tests que duplican los ya existentes.

Aislamiento de contexto entre generación y revisión

La misma sesión de Claude que generó un cambio de código conserva el razonamiento que la llevó a tomar esas decisiones, lo que la hace **menos efectiva revisando su propio trabajo** (tiende a no cuestionar sus propias decisiones). Por eso, para una revisión de calidad real conviene usar una **instancia independiente** sin ese contexto de razonamiento previo — ver también el punto 4.6 sobre revisión multi-instancia.

Evitar comentarios duplicados en revisiones sucesivas

Al volver a ejecutar una revisión tras nuevos commits, conviene incluir en el contexto los hallazgos de la revisión anterior, e instruir a Claude para que reporte **solo problemas nuevos o aún no resueltos** — evitando repetir comentarios ya hechos previamente en el mismo PR.

Resumen de antipatronos del Dominio 3

Antipatrón	Por qué falla	Alternativa correcta
Poner instrucciones de equipo en <code>~/ .claude/CLAUDE.md</code>	No se comparte vía control de versiones	Ponerlas en <code>.claude/CLAUDE.md</code> (nivel proyecto)
Un único <code>CLAUDE.md</code> monolítico con todo mezclado	Difícil de mantener, carga contexto irrelevante	<code>.claude/rules/</code> por tema, o <code>@import</code> modular
<code>CLAUDE.md</code> por directorio para convenciones de archivos dispersos (ej. tests)	No sigue archivos fuera de esa carpeta	Reglas con <code>paths</code> y patrones glob
Usar ejecución directa en tareas con implicaciones arquitectónicas	Riesgo de rehacer trabajo al descubrir dependencias tarde	Plan mode primero, ejecución directa después
Ejecutar Claude Code en CI sin <code>-p</code>	El job se queda colgado esperando input	Usar <code>-p/--print</code>
Revisar código con la misma sesión que lo generó	Menor capacidad de cuestionar sus propias decisiones	Instancia de revisión independiente

Recursos oficiales para este dominio

- Claude Code — documentación completa: <https://docs.claude.com/en/docs/claude-code>
- Slash Commands in the SDK / Agent Skills in the SDK: sección “Guides” del Developer Guide
- Agent Skills — overview, quickstart y best practices: sección “Agent Skills” del Developer Guide en <https://platform.claude.com/docs>
- Curso 1.4 *Claude Code in Action* y curso 1.1 *Introduction to Agent Skills* # 04 — Dominio 4: Prompt Engineering & Structured Output (20%)

Este dominio cubre cómo diseñar prompts que reduzcan falsos positivos, cómo usar few-shot prompting con criterio, cómo forzar salida estructurada fiable, cómo diseñar bucles de validación y reintento, y cuándo usar procesamiento por lotes.

Recursos principales: curso *Building with the Claude API*, sección “Prompt engineering” y “Structured outputs” del Developer Guide en platform.claude.com/docs.

4.1 Criterios explícitos para mejorar la precisión y reducir falsos positivos

Por qué las instrucciones vagas no funcionan

Instrucciones genéricas como “sé conservador” o “solo reporta hallazgos de alta confianza” no mejoran la precisión de forma medible, porque no le dan al modelo un criterio concreto y verificable para decidir. La alternativa que sí funciona es escribir **criterios explícitos y categóricos** — por ejemplo, en lugar de “revisa que los comentarios sean precisos”, especificar “marca un comentario solo cuando el comportamiento que afirma contradice explícitamente el comportamiento real del código”.

El coste de los falsos positivos en la confianza del usuario

Cuando una categoría concreta de hallazgos tiene una tasa alta de falsos positivos, eso erosiona la confianza del desarrollador en **todas** las categorías, incluidas las que sí son precisas. Una respuesta razonable ante esto es

desactivar temporalmente la categoría problemática mientras se mejora el prompt específicamente para ella, en lugar de dejar que contamine la percepción de todo el sistema.

Severidad con ejemplos concretos

Definir niveles de severidad (crítico / alto / medio / bajo) acompañados de ejemplos de código concretos para cada nivel produce una clasificación mucho más consistente que describir la severidad solo en términos abstractos.

4.2 Few-shot prompting para consistencia y calidad de salida

Por qué few-shot y no solo más instrucciones

Cuando instrucciones detalladas por sí solas siguen produciendo resultados inconsistentes, los ejemplos few-shot son la técnica más efectiva para conseguir un formato de salida consistente y accionable. Los ejemplos comunican de forma implícita patrones que serían muy difíciles de especificar completamente en prosa.

Manejo de casos ambiguos

Los ejemplos few-shot son especialmente útiles para demostrar **cómo razonar** en casos ambiguos — por ejemplo, mostrar por qué se eligió una tool concreta frente a otra alternativa plausible, no solo qué tool se eligió. Esto permite que el modelo **generalice el criterio de decisión** a patrones nuevos que no estaban en los ejemplos, en lugar de limitarse a repetir los casos exactos que vio.

Reducción de falsos positivos con ejemplos contrastivos

Incluir ejemplos que distingan explícitamente un patrón de código *acceptable* de un problema *genuino* reduce falsos positivos manteniendo la capacidad de generalizar — a diferencia de una lista cerrada de reglas, que solo cubre los casos exactos previstos.

Extracción de documentos con estructuras variadas

Para tareas de extracción, los ejemplos few-shot que muestran cómo manejar estructuras de documento distintas (citas en línea frente a bibliografías, secciones de metodología frente a detalles incrustados) reducen la alucinación en campos que de otro modo el modelo podría rellenar con valores inventados o dejar vacíos incorrectamente.

4.3 Salida estructurada forzada con `tool_use` y JSON Schema

Por qué `tool_use` y no simplemente pedir JSON en el texto

Usar `tool_use` con un JSON Schema como definición de entrada de la tool es el método más fiable para garantizar salida que cumpla el schema, porque **elimina los errores de sintaxis JSON** de raíz — el modelo no está “escribiendo JSON en texto libre” que luego hay que parsear con riesgo de fallo, sino generando argumentos de una llamada a función con un formato ya validado por el propio mecanismo de tool use.

`tool_choice` aplicado a extracción estructurada

- `"auto"` — el modelo puede decidir no llamar a ninguna tool y devolver texto en su lugar (no garantiza estructura)
- `"any"` — el modelo debe llamar a alguna tool, pero elige cuál. Útil cuando existen varios schemas de extracción posibles y no se sabe de antemano qué tipo de documento se está procesando.
- Selección forzada a una tool concreta (`{"type": "tool", "name": "extract_metadata"}`) — para garantizar que una extracción específica se ejecute primero, antes de pasar a pasos de enriquecimiento posteriores en turnos siguientes.

Lo que `tool_use` no resuelve por sí solo: errores semánticos

Un JSON Schema estricto vía `tool_use` elimina errores de **sintaxis**, pero no elimina errores **semánticos** — por ejemplo, que los importes de las líneas de una factura no sumen el total, o que un valor aparezca en el campo equivocado. Esos errores requieren validación adicional (ver 4.4).

Diseño de campos del schema

- Campos **opcionales/nulos** cuando la información puede no estar presente en el documento fuente — esto evita que el modelo **fabrique** un valor solo para satisfacer un campo obligatorio.
- Patrones **enum** con una opción **"other"** acompañada de un campo de texto libre para categorías extensibles que no se pueden enumerar por completo de antemano.
- Incluir reglas de normalización de formato en el propio prompt, junto al schema estricto, para manejar formato de origen inconsistente.

4.4 Validación, reintento y bucles de feedback

Reintento con feedback específico del error

Cuando una extracción falla la validación, la forma efectiva de reintentar no es simplemente “vuelve a intentarlo”, sino enviar una petición de seguimiento que incluya: el documento original, la extracción fallida, **y el error de validación específico**. Esto le da al modelo información concreta sobre qué corregir, en vez de repetir el mismo error.

Cuándo el reintento no sirve de nada

Si la información requerida simplemente **no existe** en el documento fuente (por ejemplo, está solo en un documento externo que no se ha proporcionado), reintentar no va a producir un resultado distinto por mucho feedback que se dé. El reintento es efectivo para errores de formato o de estructura de salida — no para información ausente.

Rastrear patrones de hallazgos descartados

Añadir un campo como `detected_pattern` a los hallazgos estructurados permite analizar sistemáticamente qué construcciones de código generan más descartes por parte de los desarrolladores — información valiosa para refinar los criterios del dominio 4.1.

Validación semántica con auto-corrección

Diseñar el schema para que extraiga, por ejemplo, tanto un `calculated_total` (calculado a partir de las líneas) como un `stated_total` (el que aparece literalmente en el documento), de forma que una discrepancia entre ambos se pueda marcar automáticamente con un booleano tipo `conflict_detected` — una forma de que el propio flujo detecte inconsistencias en los datos de origen sin depender de que el modelo las note por sí solo en una sola pasada.

4.5 Diseño de estrategias eficientes de procesamiento por lotes

Qué ofrece la Message Batches API

- Ahorro de costes del **50%** frente a llamadas síncronas
- Ventana de procesamiento de hasta **24 horas**
- **Sin SLA de latencia garantizado** — puede completarse antes, pero no hay garantía
- **No soporta llamadas a tools multi-turno** dentro de una misma petición del lote (no se puede ejecutar una tool a mitad de la petición y devolver el resultado dentro del mismo request)
- `custom_id` para correlacionar cada petición del lote con su respuesta correspondiente

Cuándo usarla y cuándo no

- **Apta:** cargas de trabajo no bloqueantes y tolerantes a latencia — informes nocturnos, auditorías semanales, generación de tests durante la noche.
- **No apta:** flujos bloqueantes donde alguien está esperando el resultado en tiempo real — por ejemplo, una comprobación de pre-merge que bloquea al desarrollador hasta que termina.

Un error típico de evaluación (y que aparece como pregunta de examen) es proponer mover **todo** a batch por el ahorro de coste sin distinguir qué partes del sistema son bloqueantes. La respuesta correcta casi siempre es **mixta**: batch para lo tolerante a latencia, tiempo real para lo bloqueante.

Cálculo de frecuencia de envío según el SLA

Si se necesita garantizar un SLA de, por ejemplo, 30 horas, y el procesamiento en lote puede tardar hasta 24 horas, hay que calcular una ventana de envío (por ejemplo, cada 4 horas) que deje margen suficiente para cumplir el SLA incluso en el peor caso de tiempo de procesamiento.

Manejo de fallos en lotes

Cuando algunos documentos de un lote fallan, se identifican por su `custom_id` y se **reenvían solo esos**, con las modificaciones que correspondan (por ejemplo, trocear documentos que excedieron el límite de contexto) — no hace falta reenviar el lote completo.

Optimizar antes de escalar a volumen

Antes de procesar un volumen grande de documentos por lotes, conviene **refinar el prompt sobre una muestra pequeña** para maximizar la tasa de éxito en el primer intento, reduciendo así el coste de reenvíos iterativos sobre miles de documentos.

4.6 Arquitecturas de revisión multi-instancia y multi-pasada

La limitación de la autorrevisión

Un modelo que acaba de generar código conserva el contexto de razonamiento que lo llevó a tomar esas decisiones, lo que le hace **menos propenso a cuestionar sus propias elecciones** dentro de la misma sesión. Ni instrucciones explícitas de “revisa tu propio trabajo con ojo crítico” ni el uso de extended thinking sustituyen a tener una instancia realmente independiente.

Instancias de revisión independientes

Usar una segunda instancia de Claude, **sin el contexto de razonamiento del generador**, es más efectiva detectando problemas sutiles que confiar en que la misma instancia se autocorrija.

Revisión multi-pasada para PRs grandes

Igual que en el Dominio 1 (punto 1.6), dividir una revisión de muchos archivos en pasadas por archivo (análisis local) más una pasada de integración cruzada (flujo de datos entre archivos) evita la dilución de atención y las contradicciones entre hallazgos.

Confianza auto-reportada como señal de enrutamiento

Se puede diseñar un flujo donde el modelo reporte un nivel de confianza junto a cada hallazgo, para permitir un enrutamiento calibrado de revisión — por ejemplo, priorizar revisión humana en los hallazgos de menor confianza (ver también Dominio 5, punto 5.5).

Resumen de antipatrones del Dominio 4

Antipatrón	Por qué falla	Alternativa correcta
Instrucciones vagas (“sé conservador”) para reducir falsos positivos	No dan un criterio verificable	Criterios explícitos y categóricos, con ejemplos por severidad
Confiar solo en “pide JSON” en el prompt para salida estructurada	Riesgo de errores de sintaxis al parsear texto libre	<code>tool_use</code> con JSON Schema
Asumir que un schema estricto elimina todos los errores	Solo elimina errores de sintaxis, no semánticos	Validación semántica adicional (ej. <code>conflict_detected</code>)
Reintentar indefinidamente cuando la información no está en el documento	El reintento no puede inventar datos que no existen	Detectar cuándo el reintento es inútil y devolver campo nulo
Mover todo a la Batch API por el ahorro de coste	Sin SLA de latencia, rompe flujos bloqueantes	Reservar batch para lo tolerante a latencia
Confiar en que el modelo se autorrevise con más instrucciones	Conserva su propio razonamiento y es menos crítico consigo mismo	Instancia de revisión independiente

Recursos oficiales para este dominio

- Prompt engineering — overview y técnicas: sección “Prompt engineering” del Developer Guide en <https://platform.claude.com/docs>
- Structured outputs: sección “Capabilities” → “Structured outputs” del mismo Developer Guide
- Batch processing: sección “Capabilities” → “Batch processing”
- How to implement tool use: sección “Tools” → “How to implement tool use”
- Curso 1.2 *Building with the Claude API* # 05 — Dominio 5: Context Management & Reliability (15%)

Nota: este dominio no aparecía en la estructura de 4 dominios que pediste, pero es el quinto dominio real del Exam Guide oficial (15% del examen). Se incluye para que el temario cubra el 100% del contenido evaluado.

Este dominio trata sobre cómo preservar información crítica en conversaciones largas, cuándo y cómo escalar, cómo propagar errores en sistemas multiagente, cómo gestionar contexto explorando código bases grandes, y cómo diseñar revisión humana y trazabilidad de fuentes.

Recursos principales: curso *Claude Code in Action* (para `/compact` y sesiones largas), documentación de gestión de contexto en platform.claude.com/docs, documentación del Agent SDK sobre manejo de errores.

5.1 Gestión del contexto de conversación en interacciones largas

El riesgo de resumir en exceso

Resumir progresivamente una conversación larga tiende a condensar en exceso justo el tipo de información que más importa preservar con exactitud: valores numéricos, porcentajes, fechas y expectativas concretas que el cliente ha declarado. Un resumen vago tipo “el cliente pidió un reembolso” pierde el importe exacto, la fecha del pedido o el número de referencia.

El efecto “lost in the middle”

Los modelos procesan de forma más fiable la información situada al **principio** y al **final** de una entrada larga; la información en secciones intermedias tiene más riesgo de pasarse por alto. Esto tiene implicaciones prácticas de diseño: los hallazgos clave deben colocarse al principio de una entrada agregada, con encabezados de sección explícitos organizando el resto.

El coste de acumular resultados de tools sin filtrar

Los resultados de tools se van acumulando en el contexto y consumen tokens de forma desproporcionada a su relevancia real. El ejemplo típico: una consulta de pedido que devuelve 40+ campos cuando solo 5 son relevantes para el caso concreto (por ejemplo, para gestionar una devolución). Recortar los resultados de tools a solo los campos relevantes **antes** de que se acumulen en el contexto evita ese desperdicio.

Persistencia de hechos transaccionales

Extraer los hechos transaccionales (importes, fechas, números de pedido, estados) en un bloque separado tipo “hechos del caso” que se incluye en cada prompt, **fuera** del historial resumido, evita que se pierdan al comprimir la conversación. Esto es especialmente importante en sesiones con múltiples problemas simultáneos, donde conviene mantener los datos estructurados de cada issue en una capa de contexto separada.

Ordenar los resultados de subagentes para mitigar el efecto de posición

Cuando varios subagentes devuelven metadatos (fechas, ubicación de la fuente, contexto metodológico), exigir que esos metadatos estén siempre presentes en la salida estructurada favorece una síntesis downstream más precisa — y estructurar la salida de agentes intermedios como **datos estructurados** (hechos clave, citas, puntuación de relevancia) en lugar de contenido narrativo extenso y cadenas de razonamiento reduce la carga sobre agentes downstream con presupuesto de contexto limitado.

5.2 Patrones de escalado y resolución de ambigüedad

Disparadores de escalado apropiados

- El cliente **pide explícitamente** hablar con un humano
- Hay un **vacío o excepción de política** — no necesariamente un caso “complejo” en abstracto, sino un caso donde la política simplemente no dice qué hacer
- El agente **no puede progresar** de forma significativa

Honrar la petición explícita del cliente frente a “resolver primero”

Si el cliente pide explícitamente un humano, hay que escalar de inmediato, sin intentar primero una investigación autónoma. Distinto es el caso de un cliente frustrado pero cuyo problema es sencillo y está dentro de la capacidad del agente: ahí es razonable reconocer la frustración y ofrecer resolver, escalando solo si el cliente insiste en su preferencia.

Por qué el sentimiento y la confianza autoinformada son señales poco fiables

Detectar frustración o pedir al modelo que reporte su propia confianza no correlaciona de forma fiable con la complejidad real del caso — un modelo puede estar mal calibrado y “sentirse seguro” precisamente en los casos difíciles donde más debería dudar. Ninguna de las dos señales sustituye a criterios de escalado explícitos.

Ambigüedad de identidad: varias coincidencias

Cuando una consulta a una tool devuelve varias coincidencias posibles para un mismo cliente, la respuesta correcta es **pedir un identificador adicional** para desambiguar — nunca elegir una de las coincidencias por heurística (por ejemplo, “la más reciente” o “la primera de la lista”).

5.3 Estrategias de propagación de errores en sistemas multiagente

Contexto de error estructurado

Cuando un subagente falla, la información que debe llegar al coordinador para que pueda tomar una decisión inteligente incluye: el **tipo de fallo**, la **consulta que se intentó**, cualquier **resultado parcial** obtenido, y

alternativas posibles. Un estado genérico (“búsqueda no disponible”) oculta todo el contexto que el coordinador necesitaría para decidir si reintentar, cambiar de enfoque o continuar con lo que ya tiene.

Distinguir fallo de acceso de resultado vacío legítimo

Igual que en el Dominio 2, un timeout que exige una decisión de reintento es un caso distinto de una consulta exitosa que simplemente no encontró coincidencias. Tratar ambos como “sin resultado” impide que el coordinador reaccione de forma apropiada a cada uno.

Los dos antipatrones simétricos

- **Suprimir el error silenciosamente**, devolviendo un resultado vacío marcado como éxito — esto oculta el problema en vez de exponerlo.
- **Terminar todo el flujo de trabajo** ante un único fallo — esto es excesivo cuando el sistema podría continuar con resultados parciales.

La respuesta correcta suele estar en medio: recuperación local para errores transitorios, propagación con contexto completo para lo que no se puede resolver localmente, y continuidad del flujo con anotación de las lagunas de cobertura resultantes.

5.4 Gestión de contexto en exploración de código bases grandes

Degradación del contexto en sesiones extendidas

En sesiones muy largas, el modelo puede empezar a dar respuestas inconsistentes o a referirse a “patrones típicos” genéricos en vez de a las clases concretas que ya había descubierto antes en la misma sesión. Esto es una señal de que el contexto relevante se está diluyendo o perdiendo, no un fallo aleatorio.

Scratchpads para persistir hallazgos

Mantener archivos de “scratchpad” donde el agente registra los hallazgos clave, y referenciarlos explícitamente para preguntas posteriores, ayuda a contrarrestar esa degradación — el hallazgo queda anclado en un archivo persistente en lugar de depender de que siga “vivo” en el contexto de la conversación.

Delegación a subagentes para aislar exploración verbosa

Lanzar subagentes para investigar preguntas específicas (“encuentra todos los archivos de test”, “traza las dependencias del flujo de reembolso”) mientras el agente principal se limita a coordinar a alto nivel, evita que el ruido de la exploración detallada consuma el presupuesto de contexto de la sesión principal.

Resumir entre fases

Resumir los hallazgos clave de una fase de exploración **antes** de lanzar los subagentes de la siguiente fase, inyectando ese resumen en el contexto inicial de la fase siguiente, mantiene la continuidad sin arrastrar todo el detalle acumulado.

Recuperación ante caídas con manifiestos de estado

Para sesiones largas que pueden interrumpirse, cada agente puede exportar su estado a una ubicación conocida; al reanudar, el coordinador carga un manifiesto que resume ese estado y lo inyecta en los prompts de los agentes, en lugar de tener que reconstruir todo desde cero.

/compact

El comando `/compact` reduce el uso de contexto durante sesiones de exploración extendidas cuando el contexto se satura de resultados detallados de descubrimiento que ya han cumplido su función y no necesitan permanecer completos.

5.5 Diseño de flujos de revisión humana y calibración de confianza

El riesgo de las métricas agregadas

Una precisión global del 97%, por ejemplo, puede **enmascarar** un rendimiento mucho peor en un tipo de documento o campo concreto. Una métrica agregada alta no garantiza que todos los segmentos se comporten igual de bien.

Muestreo aleatorio estratificado

Aplicar muestreo aleatorio estratificado sobre las extracciones de alta confianza permite medir la tasa de error real de forma continua y detectar patrones de error nuevos que no se habrían visto si solo se revisaran los casos que el propio sistema marca como dudosos.

Validar por tipo de documento y campo antes de automatizar

Antes de reducir la revisión humana en extracciones de alta confianza, hay que analizar la precisión desglosada por tipo de documento y por campo, para confirmar que el buen rendimiento es consistente en todos los segmentos — no solo en el promedio general.

Confianza a nivel de campo, calibrada

Que el modelo emita una puntuación de confianza por campo (no solo por documento completo) permite enrutar la atención de revisión de forma más precisa, siempre que esos umbrales de confianza se calibren usando un conjunto de validación etiquetado — no se toman como fiables “de fábrica”.

5.6 Procedencia de la información y manejo de incertidumbre en síntesis multi-fuente

El riesgo de perder atribución al resumir

Los pasos de resumido intermedios pueden comprimir hallazgos sin preservar el mapeo entre cada afirmación y su fuente original. El resultado final puede acabar afirmando algo sin que sea posible saber de dónde salió.

Mapas estructurados claim-fuente

Cada subagente debe producir salidas que asocien explícitamente cada afirmación con su fuente (URL, nombre de documento, fragmento relevante), y el agente de síntesis debe **preservar y fusionar** esos mapas al combinar hallazgos — no descartarlos en el proceso de resumir.

Datos en conflicto entre fuentes creíbles

Cuando dos fuentes fiables dan estadísticas distintas, la respuesta correcta no es elegir arbitrariamente una — es **anotar el conflicto explícitamente**, con la atribución de cada valor a su fuente, dejando que el lector (o una etapa posterior de la síntesis) decida cómo interpretarlo.

Datos temporales

Exigir que los datos estructurados incluyan la fecha de publicación o de recolección evita que diferencias temporales legítimas entre fuentes (una cifra de hace dos años frente a una actual) se malinterpreten como una contradicción cuando en realidad ambas eran correctas en su momento.

Formato apropiado por tipo de contenido

Al sintetizar, distintos tipos de contenido deben representarse en el formato que mejor les corresponde — datos financieros como tablas, noticias como prosa, hallazgos técnicos como listas estructuradas — en lugar de forzar todo a un único formato uniforme que pierde matices propios de cada tipo de dato.

Resumen de antipatronos del Dominio 5

Antipatrón	Por qué falla	Alternativa correcta
Resumir progresivamente sin cuidado	Pierde cifras, fechas y expectativas exactas	Bloque de “hechos del caso” persistente fuera del resumen
Confiar en sentimiento o confianza autoinformada para escalar	No correlaciona de forma fiable con la complejidad real	Criterios de escalado explícitos
Elegir una coincidencia de cliente por heurística	Riesgo de actuar sobre la cuenta equivocada	Pedir un identificador adicional
Devolver error genérico o suprimir el fallo silenciosamente	El coordinador no puede tomar una decisión informada	Contexto de error estructurado, con distinción fallo/vacío legítimo
Confiar solo en la precisión agregada para automatizar	Puede enmascarar mal rendimiento en un segmento	Validar por tipo de documento y campo
Descartar la atribución de fuente al resumir hallazgos	El resultado final pierde trazabilidad	Mapas claim-fuente preservados hasta la síntesis final
Elegir arbitrariamente entre estadísticas de fuentes distintas	Oculta un conflicto real de datos	Anotar el conflicto con atribución a cada fuente

Recursos oficiales para este dominio

- Context windows: sección “Build with Claude” del Developer Guide en <https://platform.claude.com/docs>
- Long context tips: sección “Prompt engineering” → “Long context tips”
- Claude Code — /compact y gestión de sesiones largas: <https://docs.claude.com/en/docs/claude-code>
- Agent SDK — manejo de errores y gestión de sesión: <https://docs.claude.com/en/api/agent-sdk/overview>
- Curso 1.4 *Claude Code in Action* # 06 — Preguntas tipo

Este archivo tiene dos bloques:

1. Las **12 preguntas de muestra oficiales** del Exam Guide, ya parafraseadas con su lógica de respuesta (recuperadas de la guía de estudio previa, para tenerlas todas juntas con el temario).
2. **8 preguntas complementarias de elaboración propia**, escritas en el mismo estilo y basadas en los mismos task statements del Exam Guide, para reforzar los dominios 4 y 5, que quedan menos representados en las 12 oficiales.

Bloque 1 — Las 12 preguntas oficiales (resumen)

Escenario: agente de soporte al cliente (Dominios 1, 2, 5)

P1. Un agente omite a veces la verificación de identidad antes de buscar un pedido, arriesgando reembolsos mal dirigidos. → *Solución:* barrera programática que bloquee `process_refund/lookup_order` hasta que `get_customer` haya devuelto un ID verificado. Un prompt no da garantía determinista.

P2. El agente confunde dos tools con descripciones mínimas y casi idénticas. → *Solución:* ampliar las descripciones (formatos, ejemplos, límites) antes que añadir few-shot o construir un router externo.

P3. Resolución en primer contacto muy por debajo del objetivo: escala casos simples, intenta resolver casos que requieren excepción de política. → *Solución:* criterios de escalado explícitos en el system prompt, con ejemplos few-shot. La confianza autoinformada no es fiable como proxy.

Escenario: generación de código con Claude Code (Dominio 3)

P4. ¿Dónde va un comando slash compartido por todo el equipo? → `.claude/commands/` en el repositorio (versionado), no en la carpeta personal del usuario.

P5. Reestructuración de monolito a microservicios, con decisiones arquitectónicas pendientes. → Plan mode, para explorar y diseñar antes de tocar código.

P6. Convenciones distintas por tipo de archivo, con tests repartidos por todo el proyecto. → `.claude/rules/` con patrones glob en el frontmatter, no `CLAUDE.md` por subdirectorio.

Escenario: sistema de investigación multiagente (Dominios 1, 2, 5)

P7. El informe final cubre solo una parte del alcance pedido, aunque cada subagente ejecutó bien su tarea asignada. → El problema está en la descomposición del coordinador, demasiado estrecha — no en los subagentes.

P8. Cómo debe fluir un timeout de un subagente hacia el coordinador. → Contexto de error estructurado (tipo de fallo, qué se intentó, resultados parciales, alternativas), nunca un estado genérico ni un vacío marcado como éxito.

P9. El 85% de las verificaciones que necesita el agente de síntesis son simples comprobaciones de hechos, y generan demasiadas idas y vueltas. → Dar al agente de síntesis una tool acotada solo para verificación simple; los casos complejos siguen pasando por el coordinador.

Escenario: Claude Code en CI/CD (Dominios 3, 4)

P10. El pipeline se cuelga esperando entrada interactiva. → Flag `-p/--print` para modo no interactivo.

P11. Se propone mover a Batch API tanto una comprobación bloqueante de pre-merge como un informe nocturno, por el ahorro de coste. → Batch solo para el informe nocturno; el chequeo bloqueante se queda en tiempo real, porque batch no garantiza SLA de latencia.

P12. Una revisión de 14 archivos a la vez da resultados inconsistentes y contradictorios entre archivos con patrones idénticos. → Dividir en pasadas por archivo + una pasada de integración cruzada, no cambiar a un modelo con más contexto.

Bloque 2 — Preguntas complementarias (dominios 4 y 5, y refuerzo de 1 y 2)

Dominio 4 — Prompt Engineering & Structured Output

Q1. Estás extrayendo datos de facturas con un schema JSON estricto vía `tool_use`. El sistema ya no produce errores de sintaxis, pero en un 8% de los casos el total calculado a partir de las líneas no coincide con el total declarado en el documento. ¿Cuál es la causa más probable y la mejor solución?

- A) El schema no es lo bastante estricto; hay que añadir más validaciones de tipo.
- B) `tool_use` con JSON Schema elimina errores de sintaxis pero no errores semánticos; hay que añadir un campo `calculated_total` junto a `stated_total` para detectar la discrepancia automáticamente.
- C) Hay que bajar la temperatura del modelo a 0 para mayor precisión numérica.
- D) Hay que cambiar a un modelo con ventana de contexto mayor.

Correcta: B. Un JSON Schema estricto garantiza forma, no corrección semántica. Detectar discrepancias exige comparar explícitamente un valor calculado contra el valor declarado, no confiar en que el modelo “sume bien” espontáneamente.

Q2. Un equipo procesa 5.000 documentos al mes con extracción estructurada. Actualmente todo va por llamadas síncronas. Quieren reducir costes. El 90% de los documentos se procesan en un informe semanal sin urgencia; el 10% restante alimenta una alerta que un analista necesita en menos de 10 minutos. ¿Qué estrategia es la más adecuada?

- A) Mover el 100% del volumen a la Message Batches API para maximizar el ahorro.
- B) Mantener todo en llamadas síncronas para evitar la complejidad de gestionar dos flujos.
- C) Mover el 90% del informe semanal a la Batch API; mantener el 10% de alertas en llamadas síncronas.
- D) Usar la Batch API para todo, con un fallback a síncrono si el lote no responde en 10 minutos.

Correcta: C. El criterio no es el volumen ni el ahorro potencial, es la tolerancia a latencia de cada flujo. El 10% de alertas es un caso bloqueante con SLA de minutos, incompatible con una API sin SLA de latencia garantizado.

Q3. Un sistema de revisión de código tiene una categoría de hallazgos de “estilo” con un 40% de falsos positivos, y una categoría de “seguridad” con menos del 5%. Los desarrolladores han empezado a ignorar también los hallazgos de seguridad. ¿Cuál es la intervención más razonable?

- A) Añadir más ejemplos few-shot a la categoría de seguridad para reforzar su precisión.
- B) Desactivar temporalmente la categoría de estilo mientras se mejora su prompt, para restaurar la confianza en el resto de categorías.
- C) Fusionar ambas categorías en una sola para simplificar la salida.
- D) Pedir al modelo que reporte solo hallazgos de “alta confianza” en todas las categorías.

Correcta: B. El problema no está en seguridad (ya tiene buena precisión) sino en que la mala categoría contamina la confianza general. Desactivarla temporalmente mientras se arregla es la intervención dirigida a la causa real.

Dominio 5 — Context Management & Reliability

Q4. Un agente de soporte gestiona una conversación larga con un cliente que ha mencionado tres importes distintos en distintos momentos. Al resumir la conversación tras 20 turnos, el agente pierde la cifra exacta del primer importe mencionado y empieza a referirse a él de forma aproximada. ¿Qué cambio de diseño soluciona esto de raíz?

- A) Aumentar la ventana de contexto del modelo.
- B) Prohibir el resumido de la conversación por completo.
- C) Extraer los importes, fechas y otros hechos transaccionales a un bloque de “hechos del caso” persistente, incluido en cada prompt fuera del historial resumido.
- D) Pedir al modelo que repita los importes en cada respuesta.

Correcta: C. El resumido progresivo tiende a difuminar justo los datos numéricos exactos. La solución no es evitar resumir (poco práctico en conversaciones largas) sino sacar los hechos críticos del proceso de resumido.

Q5. Una tool de búsqueda de clientes devuelve tres coincidencias posibles para el mismo nombre y código postal. El agente elige automáticamente la cuenta con actividad más reciente y continúa la conversación sobre esa cuenta. ¿Cuál es el problema y la corrección correcta?

- A) No hay problema real: la cuenta más reciente es estadísticamente la más probable.
- B) El agente está actuando sobre una identidad no verificada por heurística; debe pedir un identificador adicional al cliente para desambiguar.
- C) El problema es que la tool debería devolver una sola coincidencia siempre.
- D) Hay que escalar automáticamente cualquier búsqueda con más de una coincidencia.

Correcta: B. Elegir por heurística ante varias coincidencias es exactamente el antipatrón que señala el Dominio 5: el riesgo de actuar sobre la cuenta equivocada es demasiado alto como para resolverlo con una suposición. Pedir un dato adicional es la vía correcta, sin necesidad de escalar automáticamente a un humano.

Q6. En un pipeline de investigación multiagente, el subagente de análisis documental falla por un error de parseo en un PDF concreto. El coordinador recibe únicamente el mensaje "`analysis_failed`" y decide terminar todo el proceso de investigación. ¿Qué dos cosas se han hecho mal?

- A) El fallo debería haberse ignorado silenciosamente para no interrumpir la investigación.
- B) El error debería incluir contexto estructurado (qué se intentó, con qué documento, resultados parciales de otros documentos ya analizados), y el coordinador debería poder continuar con los documentos restantes anotando la laguna de cobertura, en vez de abortar todo el flujo.
- C) El subagente debería haber reintentado infinitamente hasta conseguir parsear el PDF.

- D) El coordinador debería haber cambiado automáticamente a otro modelo más grande.

Correcta: B. Es el ejemplo directo de los dos antipatrones simétricos del Dominio 5: mensaje genérico sin contexto útil, y terminar todo el flujo ante un fallo parcial y recuperable.

Refuerzo de Dominio 1 y Dominio 2

Q7. Un equipo ha implementado una regla de negocio (“nunca aprobar un reembolso superior a 1.000 €”) únicamente como una frase en el system prompt del agente. Tras varios meses en producción, se detectan tres casos en los que el agente aprobó reembolsos por encima de ese límite. ¿Cuál es la corrección adecuada según el Dominio 1?

- A) Reforzar la frase en el prompt con mayúsculas y repetirla al principio y al final del system prompt.
- B) Implementar un hook que intercepte la llamada a la tool de reembolso y bloquee programáticamente cualquier importe por encima del límite, redirigiendo a escalado humano.
- C) Reducir la temperatura del modelo para hacerlo más determinista.
- D) Añadir un mensaje de confirmación adicional que el modelo debe responder antes de aprobar.

Correcta: B. Cualquier variante de “reforzar el prompt” (A, C, D) sigue dependiendo de cumplimiento probabilístico. Una regla de negocio con impacto financiero directo necesita un hook con garantía determinista, no una instrucción por muy insistente que sea.

Q8. Un servidor MCP expone una tool `search_docs` con la descripción “busca en la documentación”. El agente, al mismo tiempo, tiene acceso nativo a `Grep`. En la práctica, el agente casi siempre usa `Grep` sobre los archivos de documentación en lugar de `search_docs`, aunque esta última hace búsqueda semántica y devuelve resultados más relevantes. ¿Cuál es la explicación más probable y la solución?

- A) El modelo prefiere por diseño las tools nativas sobre las tools MCP; no hay solución posible.
- B) La descripción de `search_docs` es demasiado genérica y no comunica su ventaja real (búsqueda semántica) frente a una búsqueda de texto literal como `Grep`; hay que enriquecer la descripción explicando sus capacidades y en qué se diferencia.
- C) Hay que eliminar el acceso a `Grep` para forzar el uso de `search_docs`.
- D) Hay que renombrar `Grep` para que suene menos atractivo al modelo.

Correcta: B. El patrón es el mismo que en el Dominio 2: cuando el modelo prefiere una tool nativa más simple sobre una tool MCP más capaz, la causa habitual es que la descripción de la tool MCP no comunica bien su ventaja — no una preferencia estructural del modelo por las tools nativas.

Cómo usar este archivo

Para cada pregunta que falles (o dudes), vuelve al apartado correspondiente del dominio en los archivos 01-05 y relee el punto exacto que sustenta la respuesta correcta — está indicado en la explicación de cada pregunta. El objetivo no es memorizar estas 20 preguntas, sino reconocer el **patrón de razonamiento** detrás de cada una, porque el examen real usará escenarios distintos pero con la misma lógica subyacente.